Math for Data Science - Lab 1

Course: Math for Data Science

This lab covers Python / NumPy / pandas basics, simple combinatorics and random sampling, Monte-Carlo simulation of the birthday problem, plotting, and the drunken-passenger problem.

Setup

First, let's make sure you have everything you need to get going. We'll use uv to manage Python and the packages for this lab:

- Install uv: https://docs.astral.sh/uv/getting-started/installation/.
- Initialize a project for this lab: `uv init` (creates a `pyproject.toml` that tracks your dependencies).
- Add the packages used in this lab: `uv add numpy pandas matplotlib jupyter`.
- Install an editor / notebook environment: VS Code or JupyterLab.
- Launch the notebook: `uv run jupyter lab` (or open this file in VS Code).

(No uv? Install Python 3.10+ directly and use `pip install numpy pandas matplotlib jupyter` instead.)

Run the cell below once to import everything we need.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from math import factorial, comb
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Exercise 1: Warmup

1. Python (with NumPy and pandas) works with a few different kinds of objects; let's get familiar with them.

a. Arrays. - i. Create an array with a single arbitrary value assigned (numerical or string). - ii. Create an empty array of length 5. - iii. Create an array with the numbers 1, 2, 3, 6, 5.

b. Data frames. Create an empty data frame with columns called x and y, both of length 5.

```
# here your code
```

2. Learn how to index and summarize your data.

a. Create an array called `nums` containing the numbers 1, 6, 7, 8, 9. b. Grab the second element of this array. c. Grab the second and third elements of this array. d. Grab all elements of this array except the first. e. Take the mean, minimum, maximum, and sum of this array. f. Find the proportion of elements of this array that are greater than 5.

> **Note:** Python indexing starts at **0**, so the "second" element is `nums[1]`.

```
# here your code
```

3. Try out some basic probabilistic functions and operations.

a. Compute 5! using `math.factorial`. b. Compute $\binom{5}{4}$ using `math.comb`. c. Use NumPy's random generator to do some random sampling, - with replacement, - without replacement, - added bonus: try to provide your own sampling probabilities.

d. Roll a die. Roll two dice. Roll a loaded die. e. Learn about seeding the random generator and try it out with your sampling. What does it do?

> **Tip:** create a generator with `rng = np.random.default_rng()` and draw samples with `rng.choice(...)`. Pass a seed - `np.random.default_rng(seed)` - to make the draws reproducible.

```
# here your code
```

Exercise 2: Simulation

4. We will now learn how to run a simulation. Recall the **birthday problem** from class. We will simulate it to see if the mathematical result from class holds up.

a. Set a seed for reproducibility. b. Start by creating one simulated class of 35 people: an array of randomly sampled birthdays. Assume all birthdays are equally likely and ignore leap years - label the days of the year 1 to 365. c. Are there any repeated birthdays in your class? Which birthdays are repeated? d. Now we will randomly generate 100,000 fake classes and compute the average probability of having at least one shared birthday (the frequentist interpretation). First, create an empty array called `match` of length 100,000 to hold our results. e. Write a loop that generates a fake class and saves the number of repeated birthdays in that class into your array. f. Summarize that output: what proportion of simulated classes have matching birthdays? g. Compare the simulated result to the mathematical result from class. Compute the probability of matching birthdays using the formula we derived. (`np.prod` may be helpful.)

```
# here your code
```

Exercise 3: Plotting

5. Now let's learn how to make plots. We'll recreate the birthday-problem curve.

a. Write a loop to compute the probability of a birthday match for $k = 1$ to 100 people. Save the output into a data frame with two columns: one for the x axis (k) and another for the y axis (probability of a match). b. Make a quick scatter plot. c/d. Improve it: add axis labels, a title, and a red horizontal line at 0.5.

```
# here your code
```

Exercise 4: The Drunken Passenger Problem

6. This is a common question in quantitative interviews. There are 100 passengers lined up to board an airplane with 100 seats. Each seat has been assigned by the airline to one of the passengers. The first passenger drunkenly sits in a **random** seat. Each subsequent passenger takes their assigned seat if it's available, and otherwise sits in a random available seat. What is the probability that the **last** passenger gets to sit in their assigned seat?

a. Create a data frame with three columns: `passenger` (1 to 100), `assigned` (their assigned seats, a random permutation), and `seat` (the seat they end up in - starts empty, gets filled during the simulation). b. Let the first passenger pick a random seat. c. Fill in the second passenger's seat, step by step: build the list of available seats, then use an `if/else`. d. Write a loop assigning seats for passengers 3 through 100 with the same logic. e. Wrap it all into a function that returns `True` if the last passenger gets their assigned seat and `False` otherwise. Run it 10,000 times. What seems to be the probability? f. Adapt the function to accept an arbitrary number of passengers. Does the number of passengers matter? g. Is there a mathematical/logical way to solve this? What is the intuition? (See the solution to Exercise 59 in Blitzstein and Hwang.)

We represent an empty seat with `-1`.

```
# here your code
```

here your solution